Project Documentation

GitHub → Jenkins → Docker → Docker Hub → AWS EC2 → Uptime Kuma / Slack

Application Deployment Project 3

Project Name	DevOps Build Deployment
Repository	https://github.com/Akashpeter19/devops-build
Cloud Platform	AWS EC2
CI/CD Tool	Jenkins
Containerization	Docker
Image Registry	Docker Hub
Monitoring Tool	Uptime Kuma
Notification Method	Slack
Dev Image	app19/devops-build-dev:latest
Prod Image	app19/devops-build-prod:latest
Application URL	http://13.232.228.100
Jenkins URL	http://3.111.246.227:8080

1. Project Overview

This project focused on building a complete DevOps pipeline for a frontend application using Docker, GitHub, Jenkins, Docker Hub, AWS EC2, and monitoring tools. The provided repository contained a ready-made frontend build, which was containerized and deployed in a professional CI/CD workflow.

The implementation followed a branch-based deployment strategy: code pushed to the dev branch triggers Jenkins to build the image, push it to the development Docker Hub repository, and deploy the application to the EC2 application server; code merged or pushed to the master branch triggers Jenkins to build the image and push it to the production Docker Hub repository.

The live deployment was also integrated with monitoring and downtime alerting so that service health could be checked continuously and incidents could be reported automatically.

2. Repository and Application Setup

The project repository was cloned from the given GitHub source. After inspection, the repository was identified as a pre-built frontend application because it only contained a build directory rather than source folders such as src, public, or package.json.

This confirmed that the correct deployment method was to serve the static frontend using Nginx inside a Docker container. The working folder was then structured with the required deployment files, scripts, Jenkins pipelines, and screenshots for submission.

3. Dockerization of the Application

A Dockerfile was created using the nginx:alpine base image. The application build files were copied into Nginx's web root so that the site could be served on port 80.

A docker-compose.yml file was added for local validation, together with two scripts: build.sh for building the Docker image as linux/amd64 and deploy.sh for running the container on port 80. This architecture choice was important because the project was developed on a MacBook, and the image had to be built in a compatible format for Linux AMD64 cloud hosts.

Local validation confirmed that the image built successfully, the container ran on port 80, and the application loaded correctly in the browser.

4. Version Control and Branch Workflow

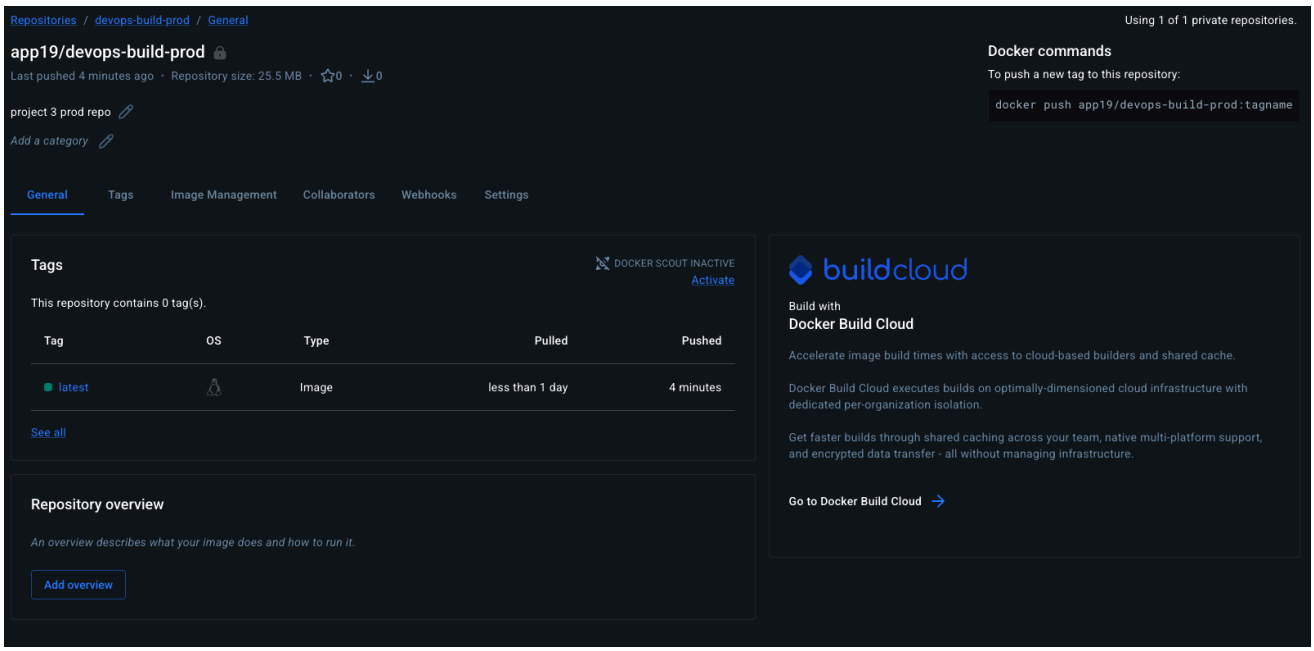
After local Docker validation, the project was pushed to a personal GitHub repository. The required Git workflow used two branches: dev for development automation and master for production image publishing.

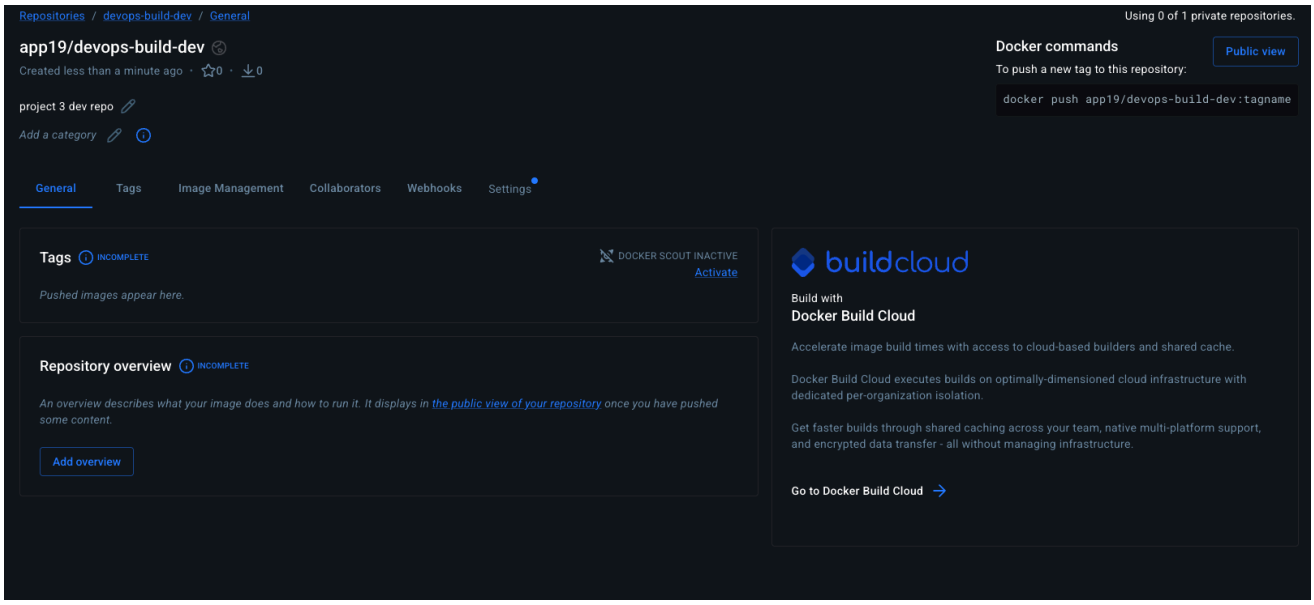
The workflow was designed so that changes pushed to dev trigger development build and deployment, while changes merged or pushed to master trigger the production image pipeline.

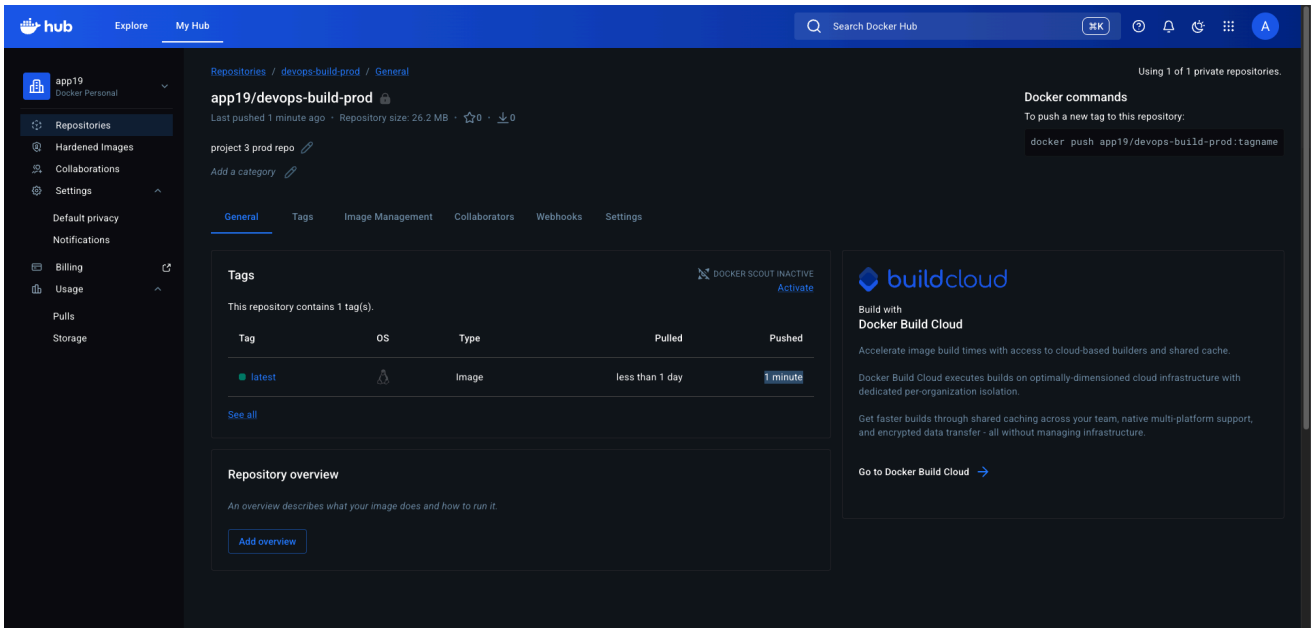
5. Docker Hub Repository Configuration

Two Docker Hub repositories were created: app19/devops-build-dev for development images and app19/devops-build-prod for production images. The development repository remained public and the production repository remained private.

Docker images were built, tagged, and pushed successfully, and Jenkins later automated the same process through the final pipeline jobs.



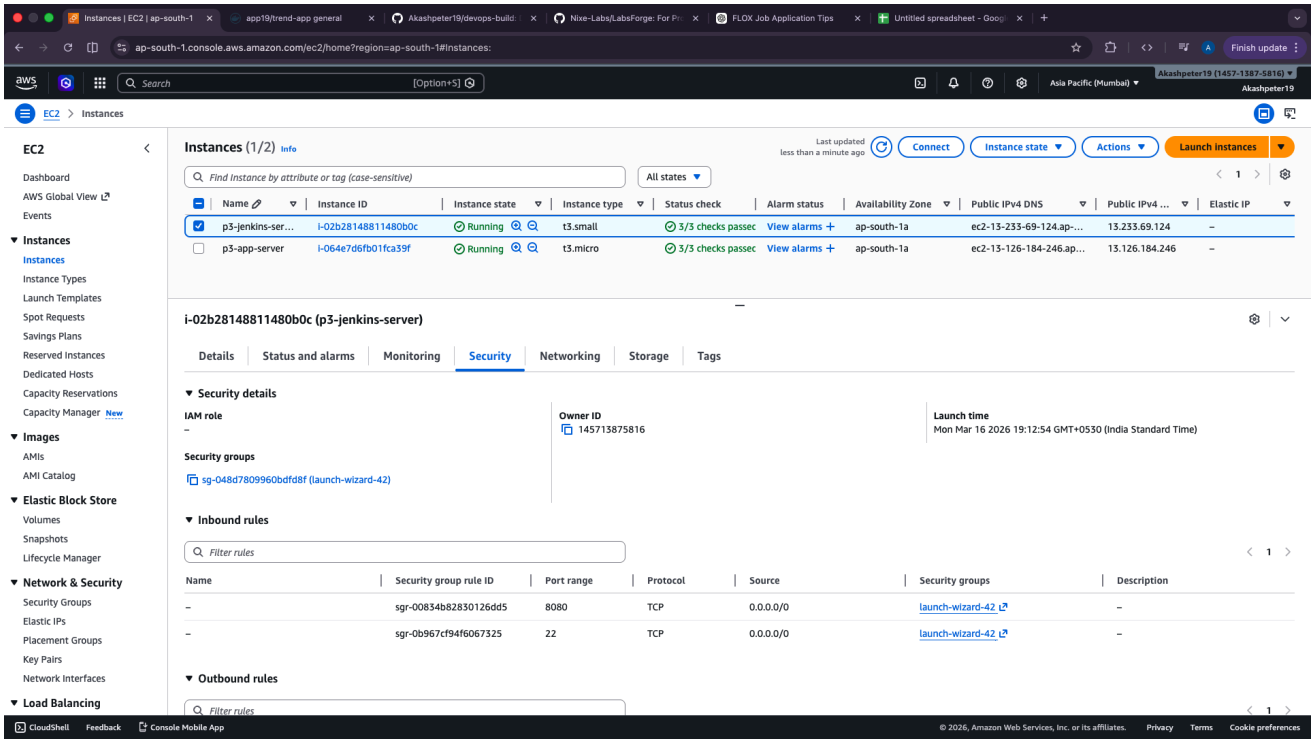


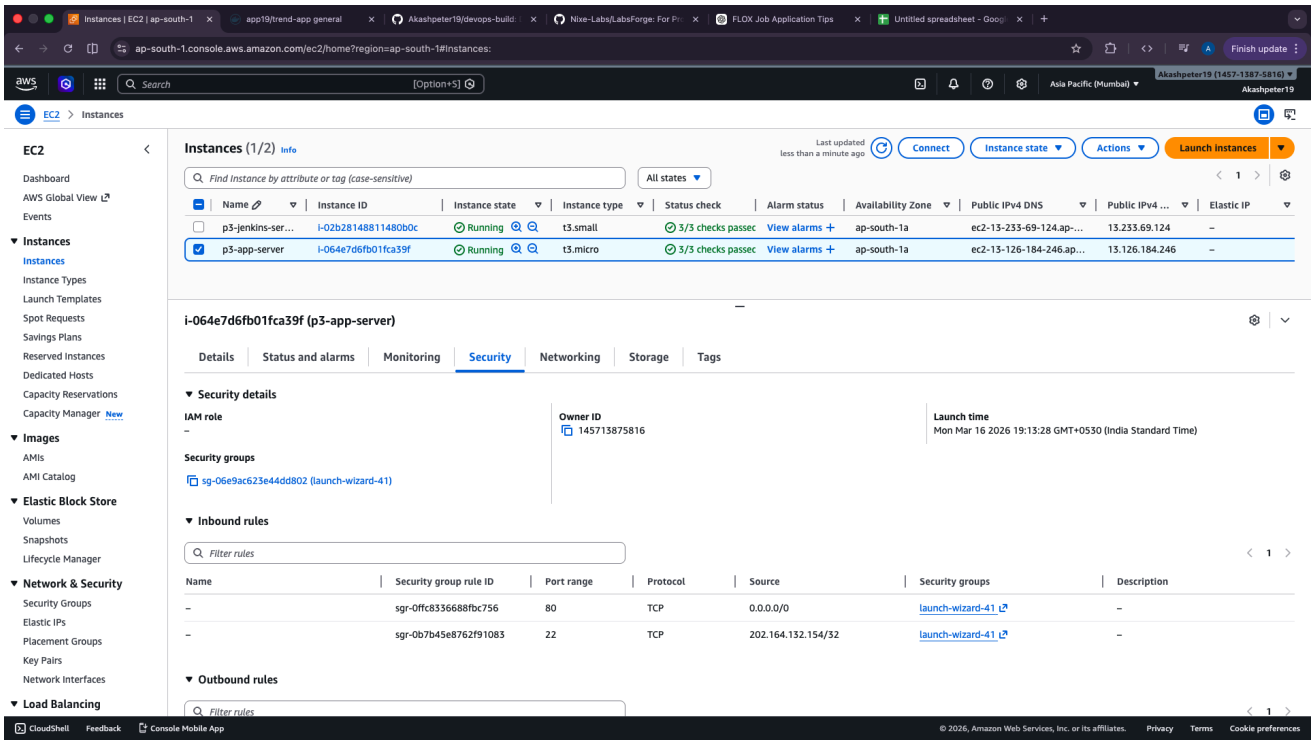


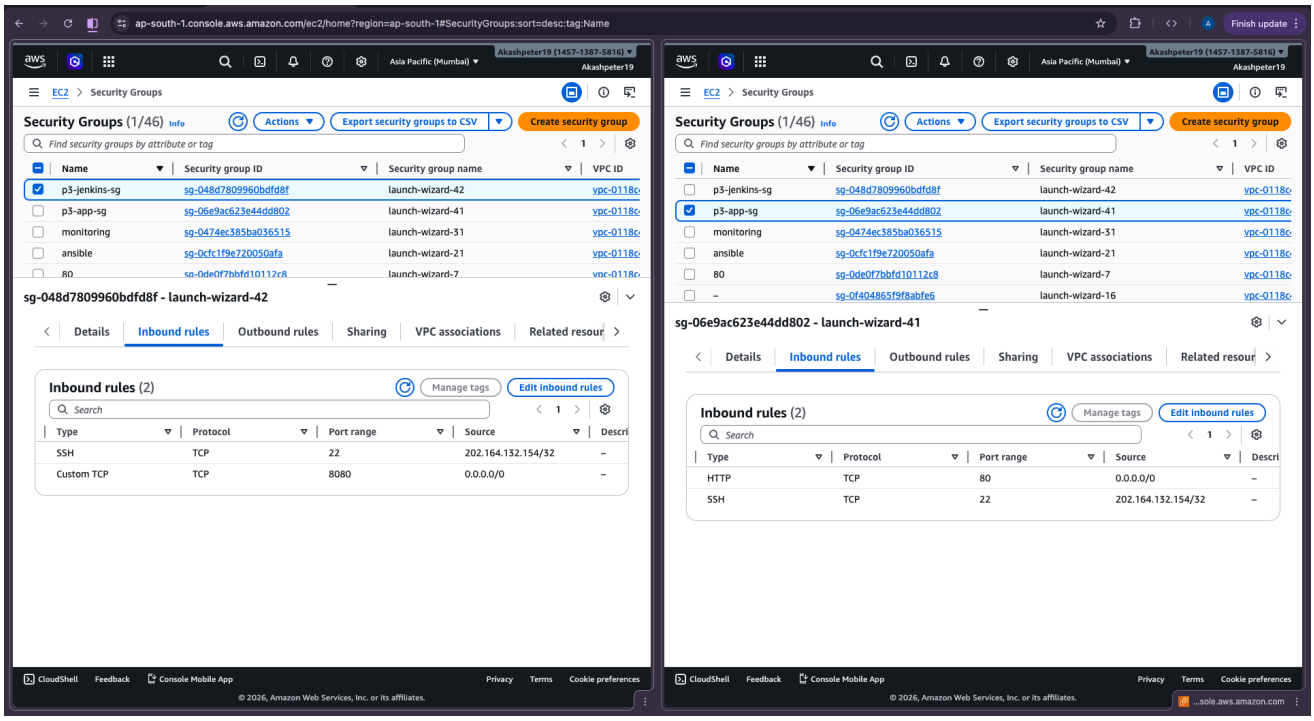
6. AWS EC2 Infrastructure Setup

Two EC2 instances were created in the ap-south-1 region to keep the architecture clean and professional. The first instance hosted Jenkins and monitoring services, while the second instance hosted the live application.

Security groups were configured carefully. SSH access was restricted to the user's IP, Jenkins UI was exposed on port 8080, Uptime Kuma was exposed on port 3001, and the application was exposed publicly on port 80. Additional SSH access from Jenkins to the application server was allowed to support automated deployment.



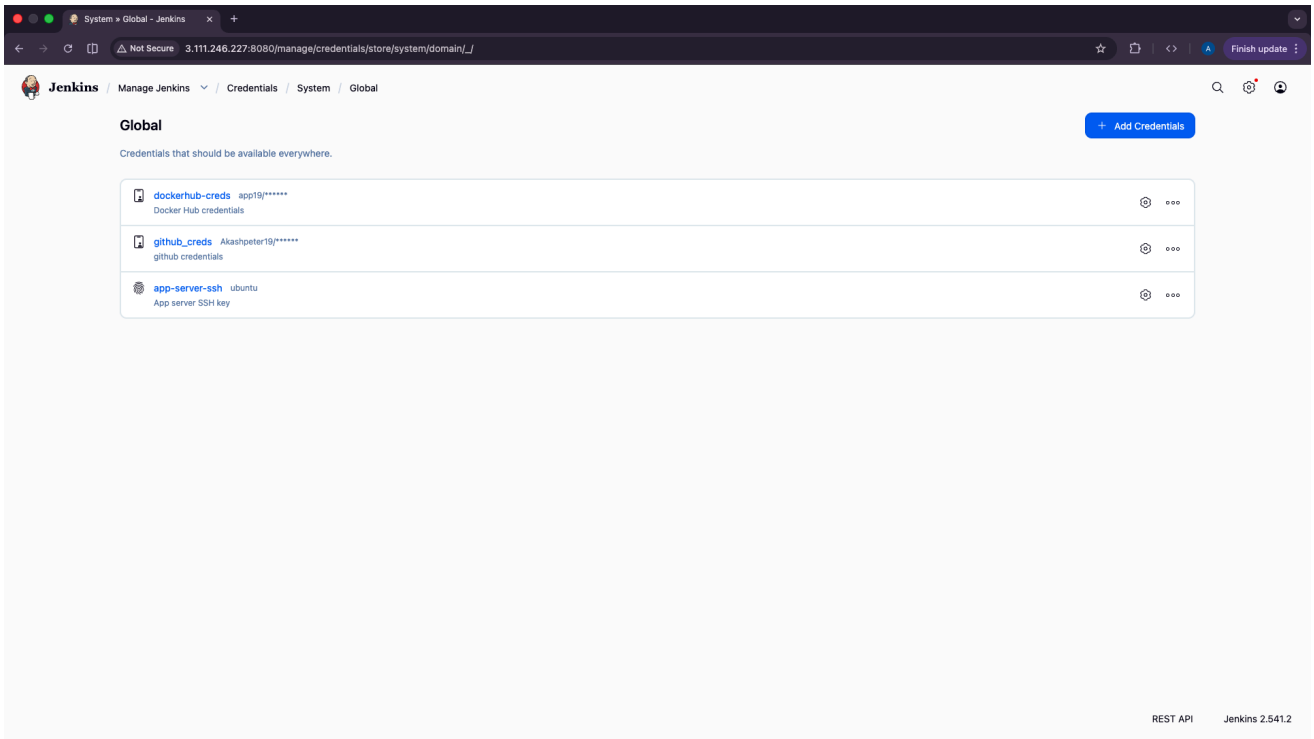




7. Jenkins Installation and Configuration

Jenkins was deployed on the Jenkins EC2 instance using Docker. Docker was installed on the server first, and Jenkins was then run as a container with persistent storage.

To support image builds and remote deployment, the Jenkins runtime was prepared with Docker socket access, Docker CLI, SSH client support, and stored credentials for GitHub, Docker Hub, and the application server SSH key.

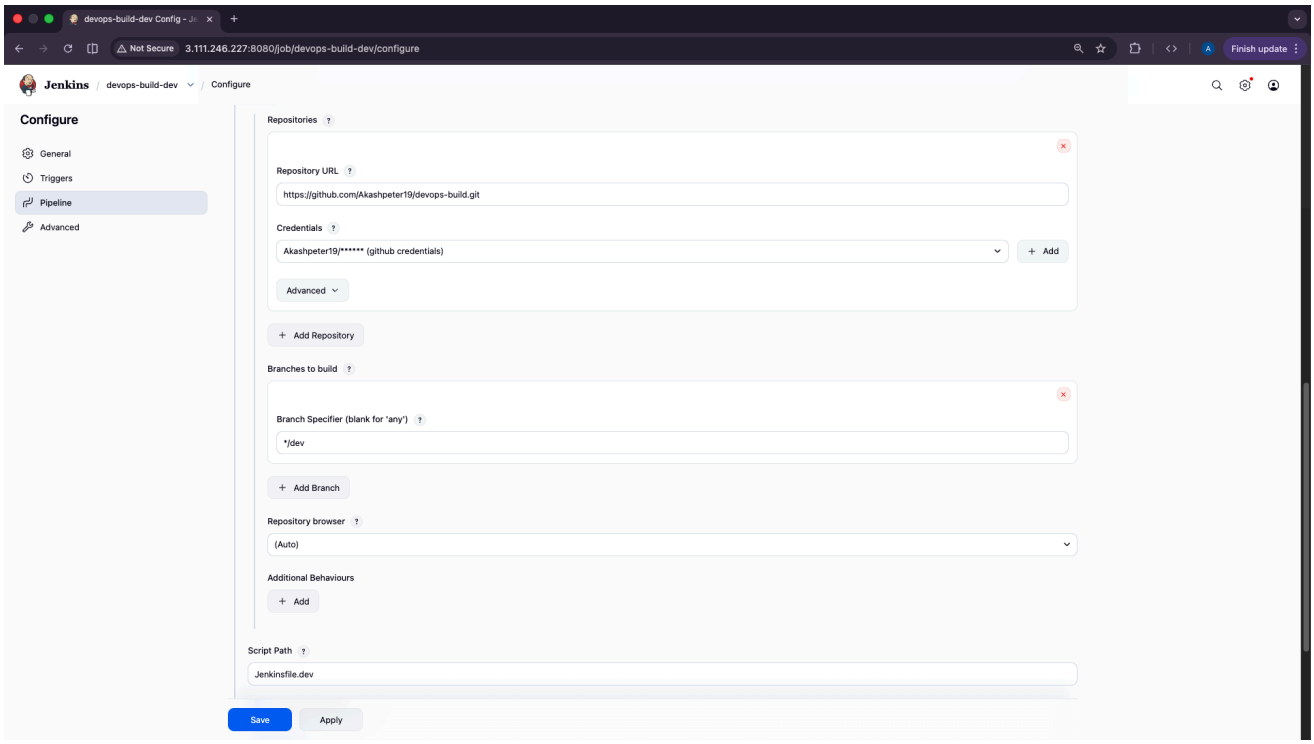


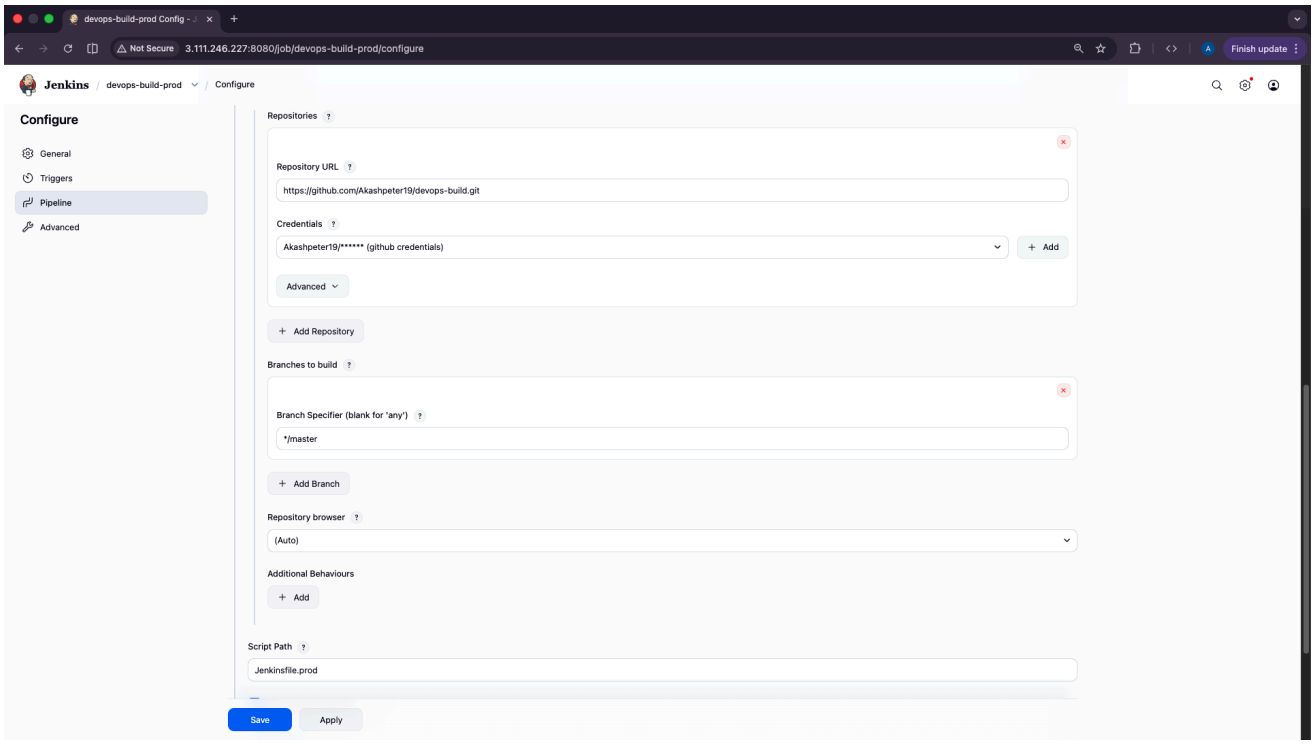
8. CI/CD Pipeline Design

To avoid manual branch switching and keep the setup fully automated, two separate Jenkins jobs were created: devops-build-dev and devops-build-prod.

The dev job is fixed to the dev branch and uses Jenkinsfile.dev. It checks out the dev branch, builds the development image, pushes it to Docker Hub, and deploys it to the EC2 application server.

The prod job is fixed to the master branch and uses Jenkinsfile.prod. It checks out the master branch, builds the production image, and pushes it to the private production Docker Hub repository.





9. GitHub Webhook Automation

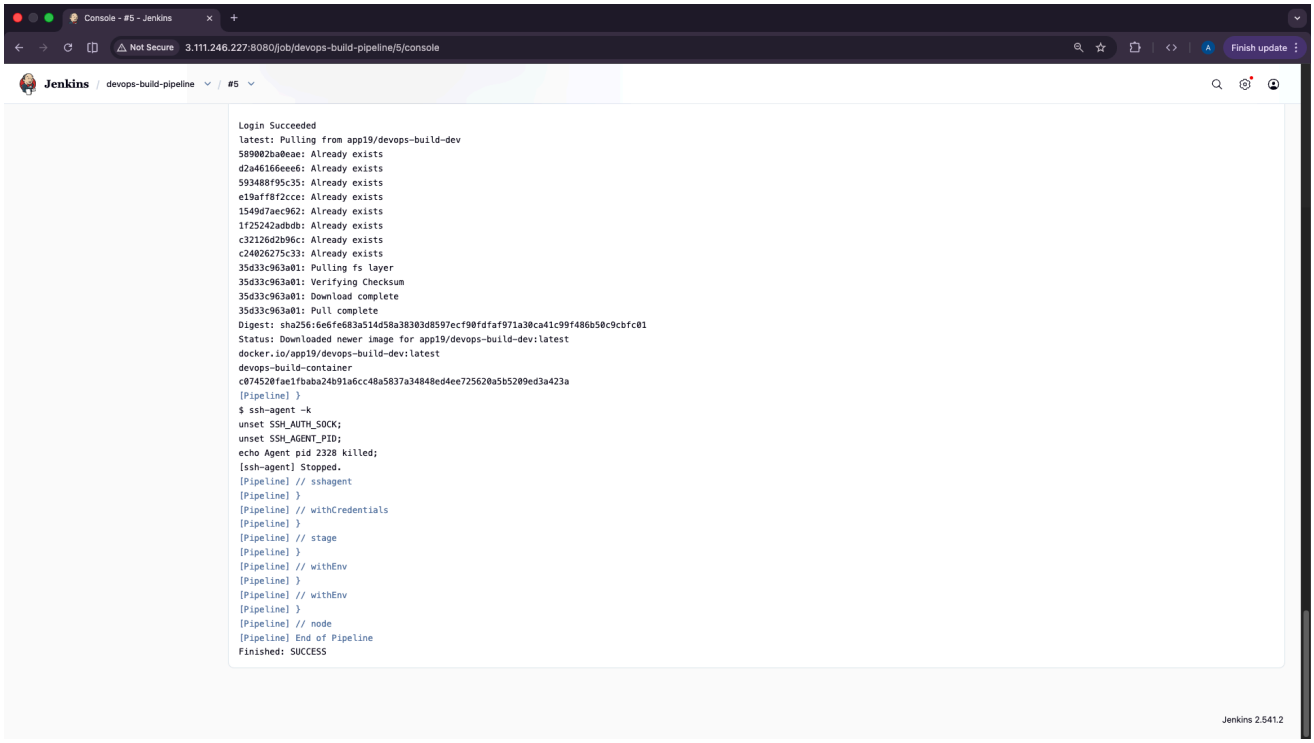
GitHub webhook integration was configured so that Jenkins automatically triggers when repository changes are pushed. The webhook target was configured to the Jenkins server endpoint, allowing the final workflow to run directly from Git activity rather than manual Jenkins interaction.

This completed the automation requirement where a push to dev starts the development pipeline, and a merge or push to master starts the production image pipeline.

10. Development Branch Automation

The development workflow was tested successfully. When code was pushed to the dev branch, Jenkins automatically triggered the development job, built the image, pushed it to app19/devops-build-dev:latest, connected to the application server using SSH, and redeployed the live container.

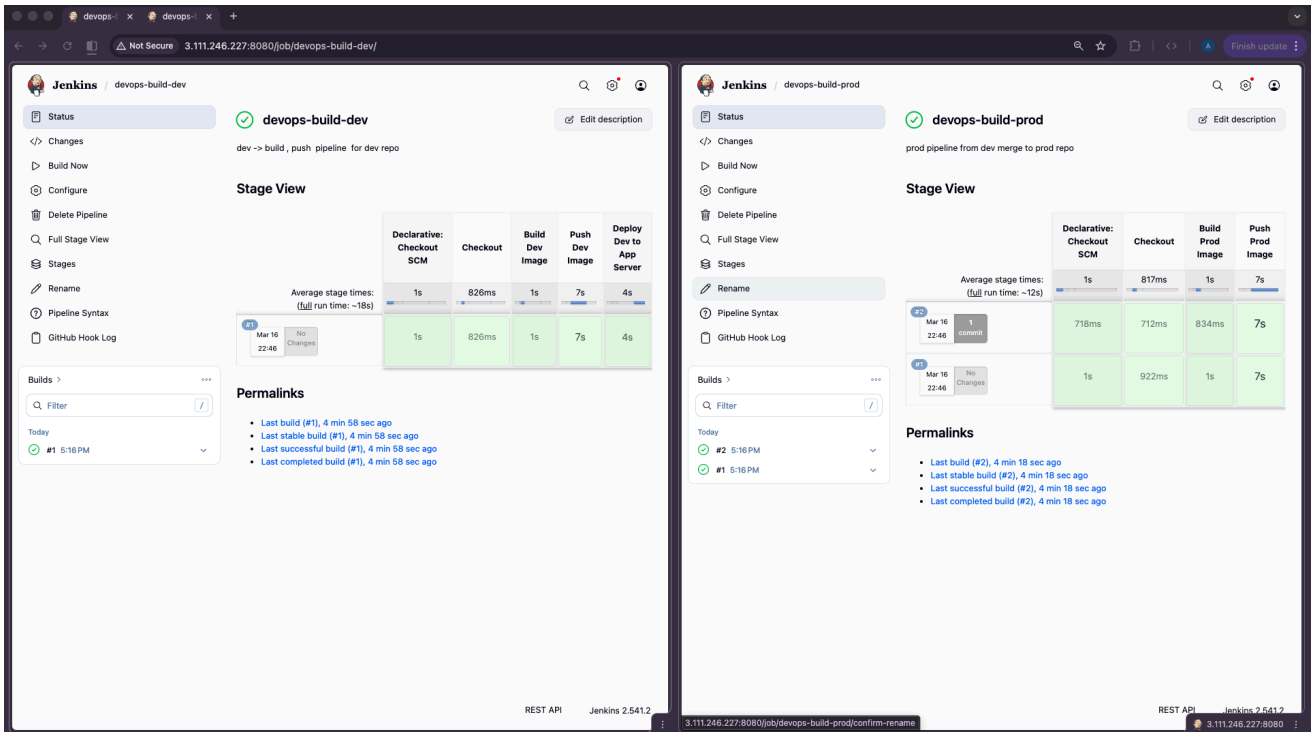
This confirmed the complete development deployment workflow from GitHub to Docker Hub to AWS EC2.



11. Master Branch / Production Automation

The production workflow was also verified successfully. When the latest code was available on the master branch, Jenkins automatically triggered the production job, built the image, and pushed it to app19/devops-build-prod:latest.

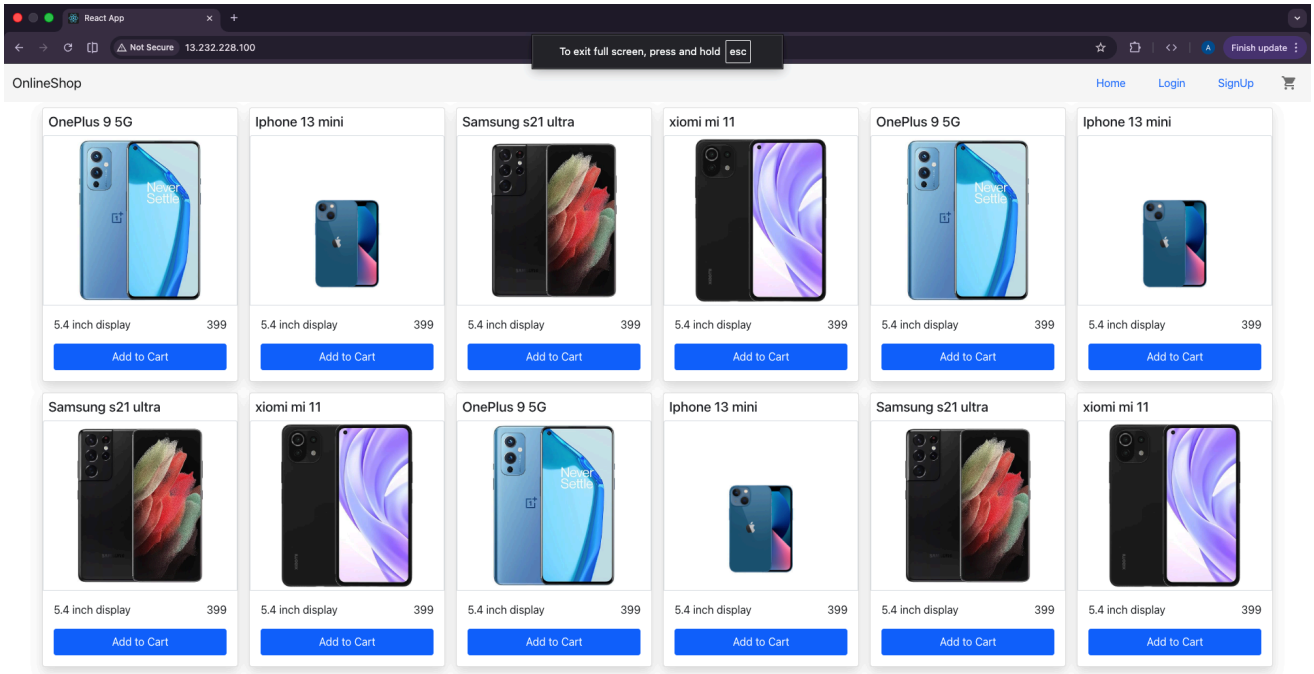
This matched the project requirement where the master branch should publish the production image to the private Docker Hub repository.



12. Live Application Deployment

The live application was deployed on the EC2 application server and exposed publicly on port 80. Manual checks and Jenkins-based redeployment both confirmed that the application was available and functioning correctly on the public EC2 IP.

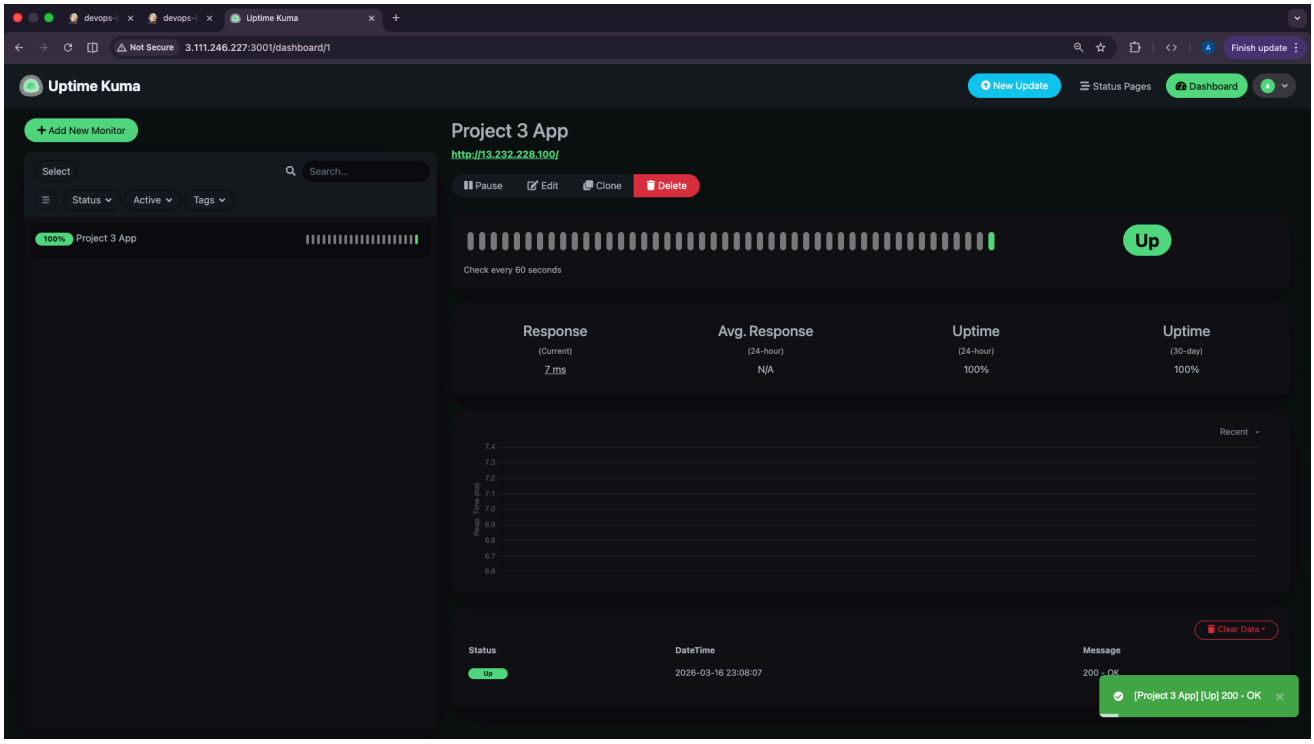
The deployed site URL used for validation was http://13.232.228.100.



13. Monitoring and Health Checks

To monitor the live application, Uptime Kuma was deployed on the Jenkins EC2 instance. A monitor was created for the application URL so that its health state could be tracked continuously.

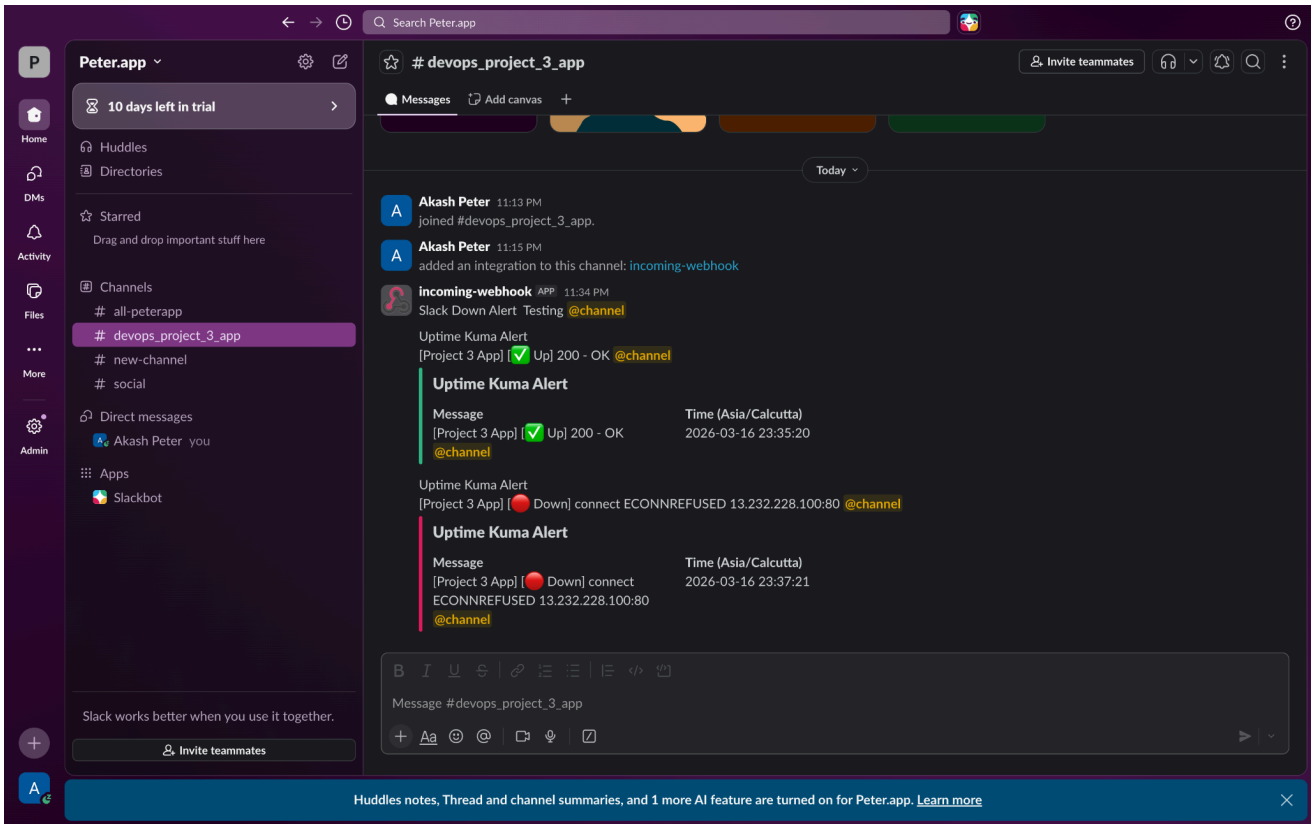
This fulfilled the requirement to monitor the health of the live application in real time.



14. Downtime Notification Testing

A Slack notification channel was integrated with Uptime Kuma to receive downtime alerts. To verify the setup properly, the application container on the app server was intentionally stopped, Uptime Kuma detected the service failure, and a Slack message was sent to the configured channel.

After testing, the application container was started again and the service returned to healthy status.



15. Final Outcome

At the end of the project, all major technical requirements were completed successfully: application containerization, local validation, GitHub branch workflow, Docker Hub repositories, Jenkins automation, AWS deployment, live application access, health monitoring, and downtime alerting.

16. Conclusion

This project successfully implemented a complete DevOps workflow for a frontend application using Docker, Jenkins, Docker Hub, AWS EC2, and Uptime Kuma.

The final setup demonstrated containerization, CI/CD automation, branch-based deployment strategy, cloud hosting, health monitoring, and alerting in a structured and professional manner.